

APP MOBILE - REACT NATIVE

Cours M1

Deschamps Marc 2025-2026



Objectifs du module



Application multiplateforme

Créer une application mobile
multiplateforme (iOS / Android)



Interface & composants

Structurer une interface avec des
composants réutilisables



États & navigation

Gérer les états avec React Hooks et
naviguer entre les écrans



APIs & stockage

Consommer des APIs et gérer le stockage local



Fonctionnalités natives

Intégrer des fonctionnalités natives (caméra, géolocalisation)

Objectifs du Jour 1



Écosystème React Native

Comprendre l'écosystème React Native



Installation Expo

Installer et utiliser Expo



Composants de base

Manipuler les composants de base



Stylisation mobile

Styliser une interface mobile



Projet MyPlaces

Créer la première version de MyPlaces

Panorama du développement mobile

Approche	Technologies	Avantages	Inconvénients
Native	Swift / Kotlin	Performance optimale	Deux codebases
Hybride	Cordova, Ionic	Production rapide	Rendu WebView
Cross-platform	React Native, Flutter	Code unique, rendu natif	Accès natif partiel

Qu'est-ce que React Native ?

Framework open source

Créé par **Facebook** en 2015

JavaScript → Natif

Écrit en JS/TS → composants natifs

Pas de DOM

Rendu via Bridge

Exemple de mapping

`<View>` → UIView / ViewGroup

Pourquoi React Native ?

Basé sur React

Composants, hooks, JSX familiers

Rendu natif

Performance réellement native

Compatible iOS & Android

Code unique, deux plateformes

Large écosystème

Communauté active et modules riches

Développement rapide

Grâce à Expo et Hot Reload



Projet fil rouge : MyPlaces

App mobile de lieux favoris :

- **Ajouter un lieu** (nom, photo, géolocalisation)
- **Afficher la liste** des lieux
- **Voir sur une carte**
- Sauvegarde locale

React vs React Native

React Web

- `<div>`
- `<p>`
- CSS
- DOM

React Native

- `<View>`
- `<Text>`
- StyleSheet
- Bridge natif

📋 Même logique, rendu différent.

L'écosystème



Expo CLI

Solution gérée pour démarrer rapidement



React Native CLI

Configuration manuelle avancée



Metro bundler

Sert le JavaScript à l'application



Expo Go

Aperçu en live sur smartphone

Le rôle d'Expo

Démarrage rapide

Configuration automatique

Test instantané

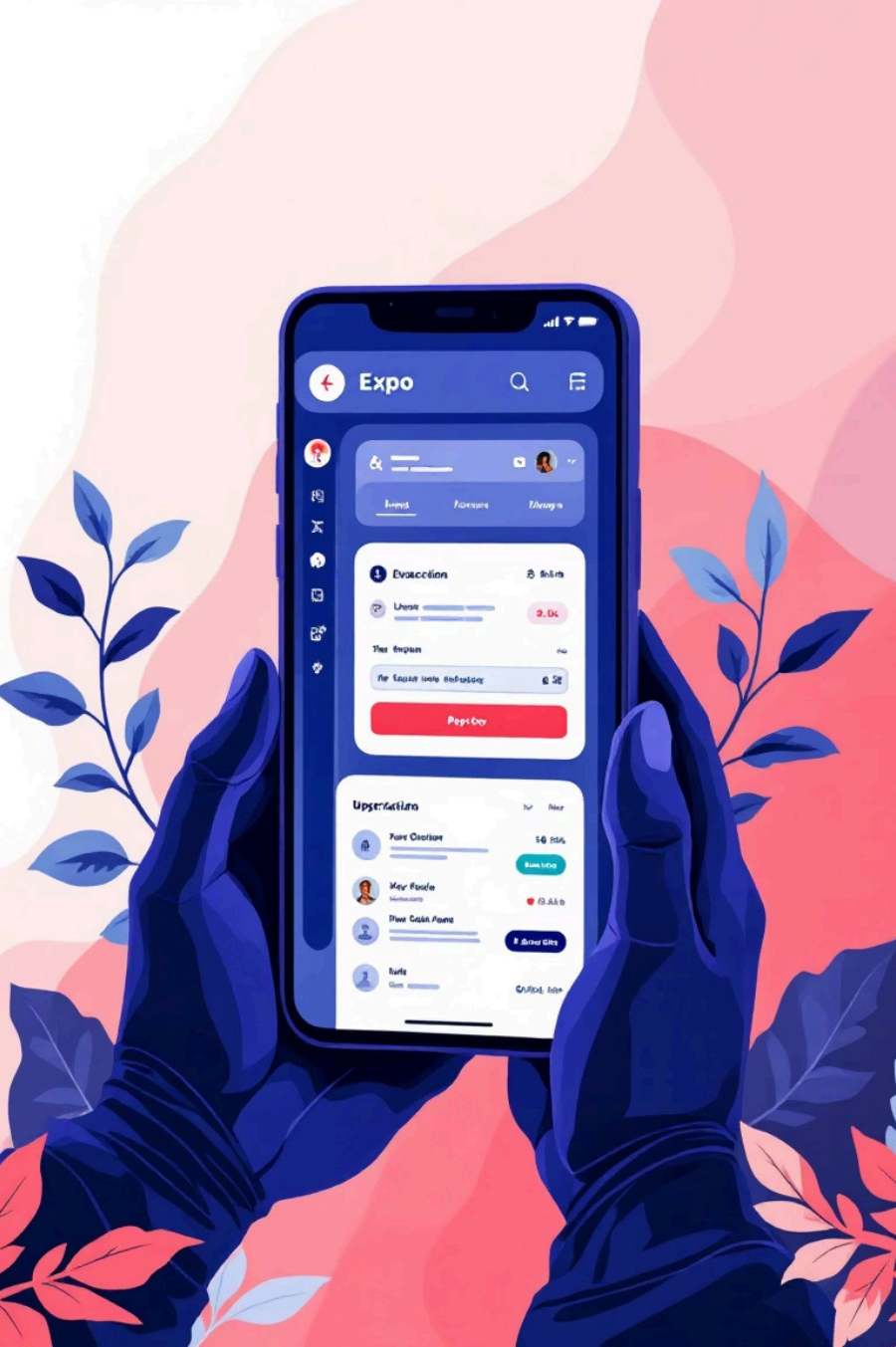
Sur mobile via Expo Go

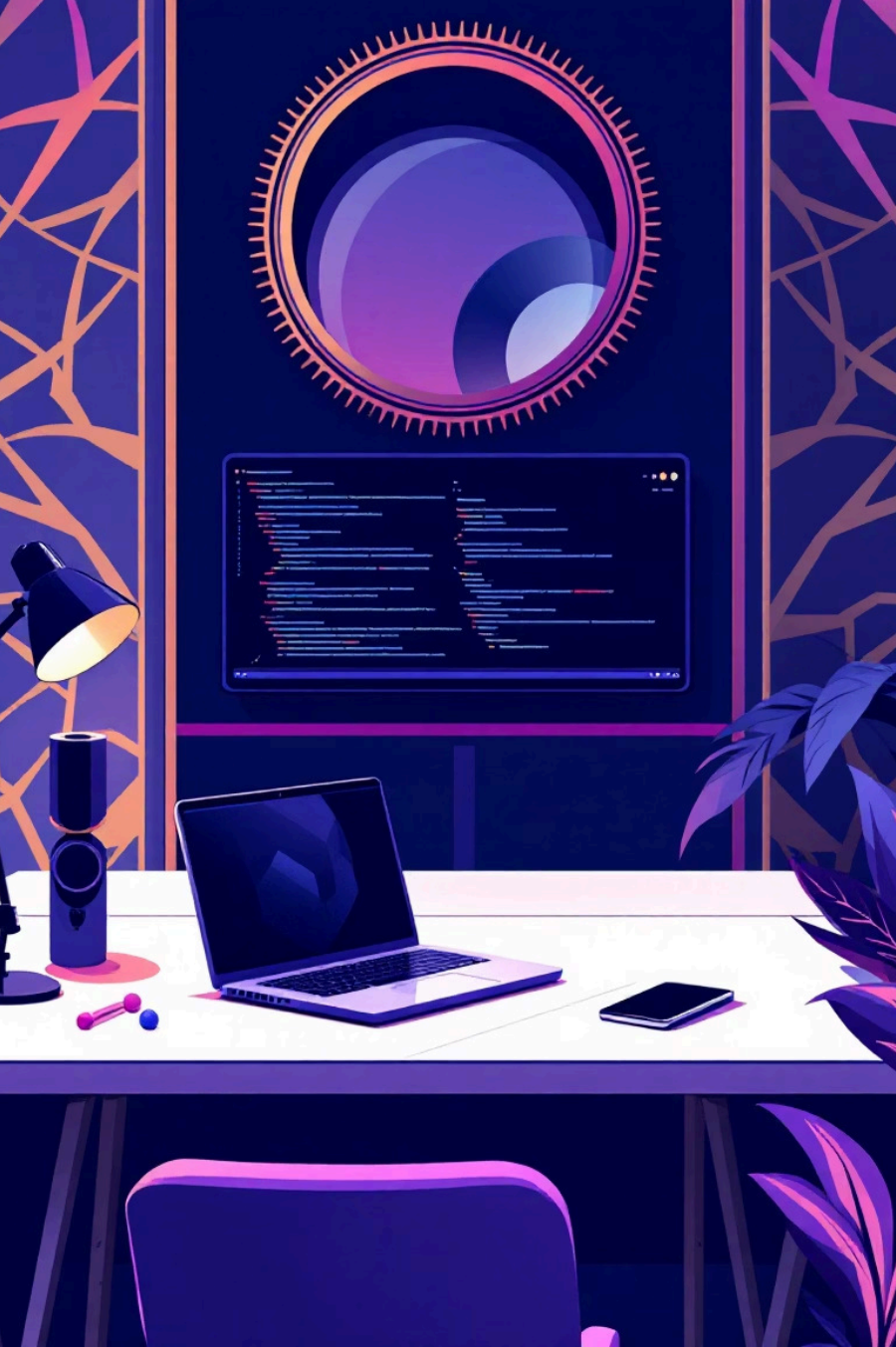
API natives simples

Intégration facilitée

Limitation

Modules non supportés → Eject





Installation de l'environnement

Pré-requis :

Node LTS / npm / VS Code / Expo Go

```
npm install -g expo-cli  
npx create-expo-app MyFirstApp  
npx expo start
```

Structure d'un projet Expo

```
MyFirstApp/  
|—— App.js  
|—— assets/  
|—— package.json  
└—— app.json
```

📄 App.js = point d'entrée.

Lancer et tester l'app



Démarrer le serveur

`npx expo start` → ouvre DevTools



Scanner le QR Code

Avec l'app Expo Go sur téléphone



Voir l'app en direct

Hot Reload = modifications immédiates !

TP 1 : Première app

Objectif

Créez votre première app React Native

Étapes

1. Créer le projet Expo
2. Modifier le texte
3. Ajouter un bouton
4. Observer le hot reload

Composants de base



View

Conteneur de base



Image

Illustration



ScrollView

Défilement



Text

Affichage texte



Button

Action



SafeAreaView

Zones protégées

Spécificités mobiles



Pas de HTML

Composants React Native uniquement



Styles → objets JS

StyleSheet au lieu de CSS



Flexbox = layout principal

Orientation verticale par défaut



Boutons natifs

Style adapté à la plateforme



Pas de classes CSS

Tout est en JavaScript

TP 2 : Interface simple

Objectif

Construis une page avec :

- Titre & sous-titre
- Bouton qui affiche une alerte

Composants à utiliser

View, Text, Button

Stylisation avec StyleSheet

Structure de base

```
StyleSheet.create({ ... })
```

Propriétés CSS

En camelCase (backgroundColor,
fontSize...)

Couleurs

#, rgb(), rgba() supportés

Flexbox mobile

Propriétés principales

`flexDirection`, `justifyContent`, `alignItems`

Orientation par défaut

Verticale (column) contrairement au web

Visuel : schéma row vs column

SafeAreaView & Dimensions

SafeAreaView

Évite le notch iPhone et zones système

Dimensions

`Dimensions.get('window')` : taille écran

TP 3 : Layout mobile

Objectif

Crée un écran centré avec :

- Logo
- Titre
- Bouton

Challenge

Joue avec Flexbox jusqu'à l'alignement parfait.

Ajouter des icônes

Bibliothèque

`@expo/vector-icons`

Exemple d'usage

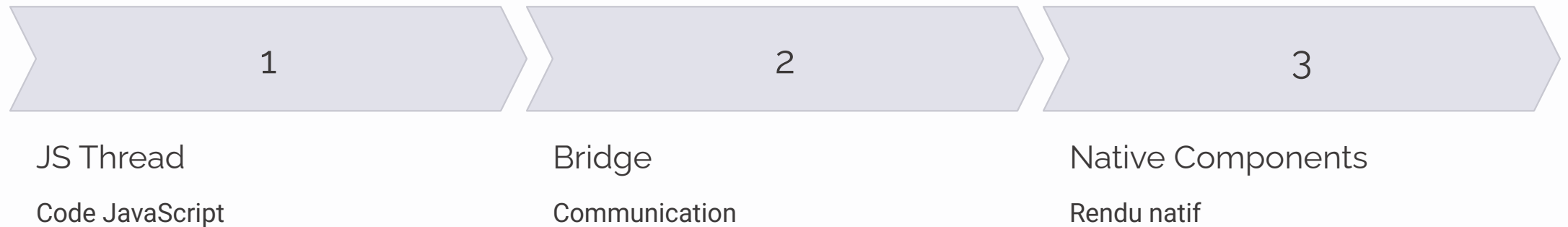
```
import { Ionicons } from  
'@expo/vector-icons';
```

Répertoire d'icônes

`icons.expo.fyi`

Sous le capot

Architecture interne :



Code JS → Bridge → Composants natifs

Rendu asynchrone = fluidité

Architecture des composants

Hiérarchie

Parent/enfant comme React

Props

Communication entre composants

Composants fonctionnels

+ hooks pour la logique

TP 4 : Composant PlaceCard

Objectif

Crée PlaceCard (titre + bouton) et affiche-le 3 fois.

Concept

Réutilisation et modularité des composants

Bonnes pratiques UI / UX

Texte ≥ 16 px

Lisibilité sur mobile

Zones tactiles ≥ 44 px

Facilité d'interaction

Marges larges

Respiration visuelle

Contrastes forts

Accessibilité

Guidelines

Respect iOS / Android

Les outils de débogage



Expo DevTools

Interface web de développement



Console.log()

Débogage simple et efficace



Flipper / RN Debugger

Outils avancés de débogage



Fast Refresh vs Live Reload

Rechargement intelligent

Astuce débogage

❏ **Erreurs JSX = écran rouge.**

Relire la console terminal pour le détail.

TP Fil rouge : MyPlaces – Partie 1

Objectif

Créer l'interface de base de MyPlaces

Fonctionnalités

- Écran d'accueil : titre + bouton
- Liste statique de 3 lieux (ScrollView)
- Composant PlaceCard

Structure du projet MyPlaces

```
MyPlaces/  
├── App.js  
├── components/  
│   └── PlaceCard.js  
├── screens/  
│   ├── HomeScreen.js  
│   └── PlacesScreen.js
```

TP Fil rouge (2/2)

Objectif

Ajoute un composant Header réutilisable.

Intégration

Intègre-le dans Home et Places Screens

Cycle de rendu et useState

```
const [count, setCount] = useState(0);  
  
<Button title="+" onPress={() => setCount(count + 1)} />  
<Text>{count}</Text>
```

❏ Seule la zone concernée est re-rendu.

Discussion

Questions ouvertes

- Différences React Web / Mobile ?
- Points difficiles ?
- Avantages d'Expo ?

Quiz rapide

1

View vs ScrollView ?
Quelle est la différence ?

2

Rôle de SafeAreaView ?
À quoi sert ce composant ?

3

npx expo start
Que fait cette commande ?

4

Qu'est-ce que le Bridge ?
Son rôle dans React Native ?

Récapitulatif Jour 1

Installation d'Expo

Environnement de développement prêt

Composants de base

View, Text, Button, ScrollView...

Stylisation Flexbox

Layout mobile et StyleSheet

Débogage

Outils et techniques

Projet MyPlaces

Première version fonctionnelle

Ressources utiles



Documentation officielle

React Native Docs



Expo Documentation

Expo Docs



React Navigation

React Navigation



Tutoriels

React Native Express



Ressources
communautaires

Awesome React Native

JOUR 2

Navigation, gestion d'état, stockage
et APIs

Objectifs : Navigation native, état global, formulaires et APIs



Navigation : comprendre la logique mobile

9h30 – 10h15

Concepts

Une app mobile $\neq$ page web : il faut une **navigation native**

Trois types de navigateurs :

- **Stack Navigator** (navigation par pile)
- **Tab Navigator** (barre d'onglets en bas)
- **Drawer Navigator** (menu latéral)

Comparatif des navigateurs

Type	UX typique	Exemple
Stack	"suivant / retour"	Détails d'un lieu
Tabs	sections principales	Accueil / Favoris
Drawer	paramètres / profil	Menu latéral

2.3 — Stack Navigator : implémentation

Installation

```
npm install @react-navigation/native  
npm install @react-navigation/native-stack  
npx expo install react-native-screens react-native-safe-area-context
```

Exemple de base

```
<NavigationContainer>  
  <Stack.Navigator>  
    <Stack.Screen name="Home" component={HomeScreen}/>  
    <Stack.Screen name="Details" component={DetailsScreen}/>  
  </Stack.Navigator>  
</NavigationContainer>
```

Stack Navigator : fonctionnalités

Passage de données

```
navigation.navigate('Details', { id: place.id });  
const { id } = route.params;
```

Customisation d'en-tête

```
options={{  
  headerStyle:{backgroundColor:'#0066cc'},  
  headerTintColor:'#fff'  
}}
```

TP 1 — Navigation Stack

Objectif

Créer une navigation entre :

- HomeScreen (liste des lieux)
- AddPlaceScreen (formulaire d'ajout)
- DetailsScreen (détails d'un lieu)

Tâches

1. Configurer un Stack Navigator dans App.js
2. Ajouter les 3 écrans
3. Naviguer de Home → AddPlace, puis revenir

2.4 — Tabs & Drawer

11h15 – 12h00

Tab Navigator

```
npm install @react-navigation/bottom-tabs
```

```
<Tab.Navigator>  
  <Tab.Screen name="Mes Lieux" component=  
    {HomeScreen}/>  
  <Tab.Screen name="Paramètres" component=  
    {SettingsScreen}/>  
</Tab.Navigator>
```

Drawer Navigator

```
npm install @react-navigation/drawer
```

```
<Drawer.Navigator>  
  <Drawer.Screen name="Accueil" component=  
    {HomeScreen}/>  
  <Drawer.Screen name="Profil" component=  
    {ProfileScreen}/>  
</Drawer.Navigator>
```

Quand choisir chaque type de navigation ?

Stack

Pour navigation verticale

Tabs

Pour sections principales

Drawer

Pour options secondaires

TP 2 — Navigation combinée

12h00 – 12h30

Objectif

Ajouter une barre d'onglets en bas :

- "Mes lieux"
- "Ajouter un lieu"
- "Profil"

Technique

Intégrer le Stack dans un Tab Navigator

2.5 — useState & useEffect avancés

Concepts

- useState pour l'état local
- useEffect pour gérer le cycle de vie

Exemples concrets

- Charger des données au démarrage
- Mettre à jour un compteur ou un statut
- Nettoyer un timer

Démo :

```
useEffect(() => {  
  console.log("Composant monté");  
  return () => console.log("Démonté");  
}, []);
```

2.6 — Gestion d'état globale avec Context API

Problème

Le "prop drilling" : on passe trop de props entre composants

Solution

Utiliser createContext + useContext

Implémentation complète :

```
const PlacesContext = createContext();

export function PlacesProvider({ children }) {
  const [places, setPlaces] = useState([]);
  const addPlace = (p) => setPlaces(prev => [...prev, p]);

  return (
    <PlacesContext.Provider value={{ places, addPlace }}>
      {children}
    </PlacesContext.Provider>
  );
}
```

Context API : utilisation dans App.js

```
<PlacesProvider>  
  <NavigationContainer>...</NavigationContainer>  
</PlacesProvider>
```

TP 3 :

Implémente un PlacesContext et utilise-le dans Home & AddPlace.

2.7 — Gestion des formulaires

Composants principaux

- TextInput
- Switch
- Pressable
- KeyboardAvoidingView

Exemple :

```
<TextInput
  style={styles.input}
  placeholder="Nom du lieu"
  value={title}
  onChangeText={setTitle}
/>
```

Validation manuelle :

```
if(!title.trim()) Alert.alert("Erreur", "Le titre est obligatoire");
```

📌 Bonus : présenter brièvement react-hook-form

2.8 — Stockage local

AsyncStorage

```
npx expo install @react-native-async-storage/async-storage
```

```
await AsyncStorage.setItem('places', JSON.stringify(places));  
const saved = await AsyncStorage.getItem('places');
```

TP 4 :

Sauvegarder automatiquement les lieux à chaque ajout et les restaurer au démarrage.

2.9 — Consommation d'API REST

Exemple :

```
useEffect(()=>{  
  const fetchPlaces = async () => {  
    const res = await fetch('https://fakestoreapi.com/products');  
    const data = await res.json();  
    setPlaces(data);  
  };  
  fetchPlaces();  
},[]);
```

Concepts

- Promises / async-await
- ActivityIndicator
- try/catch pour les erreurs

TP 5 :

Crée un écran "Explorer" qui affiche des lieux issus d'une API.

2.10 — Hooks personnalisés

```
export function useAsyncStorage(key, initial) {  
  const [value, setValue] = useState(initial);  
  
  useEffect(()=>{  
    AsyncStorage.setItem(key, JSON.stringify(value));  
  },[value]);  
  
  return [value, setValue];  
}
```

📌 Utiliser ce hook pour gérer la persistance dans toute l'app

2.11 — Architecture et bonnes pratiques

Structure professionnelle

```
src/  
├── components/  
├── screens/  
├── context/  
├── hooks/  
└── services/
```

Règles de base

- 1 composant = 1 rôle
- Centraliser la logique dans les hooks ou context
- Gérer les couleurs et styles globalement
- Commencer à penser scalable

Synthèse du Jour 2

Navigation complète

Stack / Tab / Drawer

Gestion d'état avancée

Context API et hooks

Formulaires et validations

TextInput et gestion des erreurs

Stockage local

AsyncStorage pour la persistance

API externe

Fetch et gestion asynchrone

Architecture propre

Structure modulaire et hooks

Livrables de fin de journée

Projet MyPlaces interactif

- Lieux ajoutés, persistants et affichés dynamiquement
- Navigation fluide entre écrans

(Optionnel) Écran "Explorer"

Connecté à une API externe pour découvrir de nouveaux lieux